A perfect flow field can be a bad initial condition:
representation, not accuracy, decides neural warm starts for RANS

Ali Jabbary[a,*], Kasra Ghanavati[b]

[a] *Department of Mechanical Engineering, Urmia University, Urmia, Iran*

[b] *School of Computing and Mathematical Sciences, University of Greenwich, London, United Kingdom*

Abstract

Neural surrogates for external aerodynamics are usually evaluated as predictors. Used instead as initial conditions for a production RANS solver they are routinely worse than no initialisation at all, and the cause is the surrogate's output representation rather than its accuracy. We show this on the exact converged flow field, so no property of any network enters. Read at the solver's own cell centres it saves 92.0% of a cold solve; stored first on an equal-budget wall-fitted grid it costs 181.6% (95% CI -318 to -45, winning 1 case of 5); stored on a uniform 128^2 raster, 548%. A 274-point swing from representation alone. The obvious explanation is that a grid cannot hold the near-wall state, and we quantify that damage in closed form – every cell nearer the wall than the representation's first station receives that station's velocity, so the first-cell gradient is overestimated by u+(y1+)/u+(yc+), a ratio fixed by the law of the wall, which bounds the measurement above across a fifty-fold range of stations and from Re 1e3 to 3e6. Placement, not budget, controls it: moving the first station inside the first cell takes the gradient error from 1219% to 1.8% at half the stored values. **But that is not what costs the solve.** The arm reproducing the near-wall state to 1.8% converges at -266.8%, winning 0 of 5, worse than the arm at 1219%. Every projection preserves viscous drag (+69 to +86%) and destroys total drag, locating the damage in the pressure field.

—

Keywords: warm start, Reynolds-averaged Navier–Stokes, neural operator, convergence, acceleration, boundary layer, OpenFOAM

Highlights

- A perfect flow field, stored on a 128^2 grid, is a worse RANS start than freestream
- The same converged field: +92.0% read natively, -181.6% stored on a grid first
- Placement not budget: first station inside the first cell, 1219% error to 1.8%
- Yet the near-perfect near-wall seed converges worst of all, winning 0 of 5 cases
- Projections preserve viscous drag and destroy total drag: the damage is pressure

*Corresponding author.

Email address: st_a.jabbary@urmia.ac.ir (Ali Jabbary)

1. Introduction

A neural surrogate that predicts a flow field in 30 ms is not, by itself, a faster CFD pipeline. Engineering decisions are made on force coefficients with error bars, and a surrogate whose drag is 5% off does not replace a solve — it precedes one. The natural role for it is therefore as an **initial condition**: if the surrogate's field is closer to the answer than a uniform freestream, the solver should need fewer iterations to get there, and the saving is free of any accuracy claim, because the converged answer is the solver's, not the network's.

That argument is clean and, as stated, wrong. This paper is about why, about the one number that decides it, and about what to do when that number comes out badly.

Our starting configuration is deliberately unfavourable to shortcuts. The solver is unmodified OpenFOAM v2606 [11] `simpleFoam` with the Spalart–Allmaras model [10]. The mesh is a wall-resolved body-fitted C-grid, 31,700 cells, first cell centre $5{\cdot}10^{-6}$ chords off the wall, aspect ratios up to $2{\cdot}10^{5}$. The Reynolds number is $3{\cdot}10^{6}$ — flight, not a demonstration. The surrogate is a Transolver-style point model trained on AirfRANS, evaluated on airfoils and angles it did not see. The metric is not a residual threshold: it is the number of iterations before a force coefficient enters and stays inside a band around its converged value, because that is the quantity an engineer actually waits for.

The failure, with accuracy removed from the experiment

Take the **exact converged flow field** — not a prediction, the answer itself — store it the way neural-operator outputs are normally stored, and hand it back to the solver as an initial condition. On a 128^2 Cartesian raster, total-drag convergence is **548% slower** than starting from uniform freestream. On an equal-budget wall-fitted 256×64 grid, **173% slower**. A perfect answer, stored in the field's standard output format, is a *worse* initial condition than no initialisation at all.

Because the field being stored is exact, nothing about this depends on how accurate any network is, how it was trained, or which architecture it uses. The defect is in the **representation**.

It is not resolution, and the arithmetic says so

The obvious response is to spend more values. We measured that too — a ladder of uniform rasters from 128^2 to 421^2 of the same exact field — and the saving is flat and negative throughout (§7.1).

The reason is that the failure is not one of resolution but of **placement**, and it is arithmetic rather than statistical. When a field is resampled through a grid, every mesh cell nearer the wall than the grid's first wall-normal station receives the value belonging to that station; the representation holds no sample in between. Viscous drag is the integral of the tangential velocity gradient in the first cell — 60–84% of total drag in these cases — so the reconstructed gradient is not degraded but *replaced*, and overestimated by

$$\mathbf{G = u^{+}(y_1{}^{+}) \,/\, u^{+}(y\_c^{+})}$$

where `y1+` and `y_c+` are the wall-unit positions of the representation's first station and the mesh's first cell centre. This is the law of the wall and nothing else; **it contains no fitted parameter**. Measured over five cases at five first-station heights spanning a factor of fifty, it **bounds the damage above** in every row, by between 1.3× and 2.6× (§6.2).

Two things follow immediately. `u+` grows *logarithmically*, so a 10.8-fold increase in stored values moves the predicted damage from 36.6× to 35.3× — which is why the resolution ladder is flat, and why it is flat by a computable amount. And the fix is a grading choice rather than a budget: a 64-level geometric stack whose first station lies inside the first cell needs a growth ratio of 1.214, an ordinary mesh. **A 512^2 raster holds 262,144 values and still fails; a wall-fitted grid of 8,192 values passes** (§6.5).

Contributions

Ordered by what we think survives, not by what is largest.

1. **A representational failure of the field's standard output format, measured with the accuracy confound removed** (§1, §7.1). The exact converged answer, stored as a raster, is a catastrophically bad initial condition, and refining the raster cannot fix it.
2. **A closed-form criterion for it, with no fitted parameter**, computable from a mesh and an output format before any solve is run, and a measured *upper bound* on the damage across a fifty-fold range of first stations (§6.2). We ship it as a command-line tool so it can be run on a mesh and a format that have nothing to do with this study (§6.4).
3. **The criterion holds across Reynolds number**, checked from Re 10^3 to $3{\cdot}10^6$ at no compute cost, including its counterintuitive prediction that the same representation on the same mesh does *more* damage at lower Reynolds — and a measured statement of the regime where it stops being quantitative (§6.6).
4. **Three independent demonstrations that the near-wall state is not the mediator** (§5.2.1, §5.5, §6.7), and a located alternative (§5.2.2). Moving the representation's first station inside the mesh's first cell takes the gradient error to **1.8%** and the roughness to the converged field's own — and convergence gets *worse*, winning 0 of 5. Repairing the gradient by wall function moves convergence 0.1 points; smoothing that repair moves it 0.6 more. Meanwhile every projection preserves viscous drag and destroys total drag, locating the damage in the **pressure** field. We predicted the opposite, in writing, before running any of it.
5. **A recipe with three necessary conditions**, each isolated by a controlled arm changing one variable on one prediction, none sufficient alone (§5.2–§5.4), and **generality at n = 13** — +18.4% on viscous drag, 13/13 cases, p = 0.0002, with a passing oracle control and a null negative control (§5.1).
6. **A comparison with the classical warm start** the machine-learning initialisation literature does not make: grid sequencing, with its coarse solve charged (§5.7). It makes a **better seed than ours** — +75.9% on viscous drag against +14.6%, exactly as the criterion predicts for a coarsened body-fitted mesh — and still loses, because the coarse solve costs 1486 fine-equivalent iterations against a cold run of 696. **The learned seed's advantage is price, not quality.**
7. **An acceptance certificate** bounding the worst case at (1 + K/N) × cold with a rule that never sees a cold run (§8), and **three falsified predictions** a reader would otherwise make (§7).

What this paper does not claim

It does not claim a speed record, and it is not competing for one. The largest number in this literature is 26.3× [8]; ours is +18.4%. §2.1 and §7.3 explain by measurement rather than rebuttal why that number and ours concern different regimes — an oracle seed of the entire wake buys +0.5% here, and their own result is reported as conditional on an accurate near-body field supplied separately. A reader who wants the fastest warm start should read that paper; a reader who

wants to know whether *their* surrogate can warm-start *their* solver, and to find out before paying for it, should read this one.

It does not claim the surrogate is accurate — accuracy is a separate question and irrelevant here, since the solver converges to its own answer regardless. And it does not claim generality of the *demonstration* beyond 2-D incompressible steady RANS with one turbulence model on NACA 4-digit sections. The **criterion** is more general than that by construction, since it is a statement about wall units and sampling positions rather than about any solver or model, but we measured its consequences in one place and §10 says so plainly.

2. Related work

2.1 Warm-starting a flow solver with a learned field

Using a network's prediction as a solver's initial condition is an active line with a wide spread of reported gains, from about 2× to 26×. The spread is usually read as a difference in method quality. **We read it as a difference in representation and in which region is seeded, and §6 gives the criterion that sorts it.**

Zhou et al. [12] map a low-fidelity potential-flow solution to a RANS field with an equivariant vector-cloud operator and use the result to start `rhoSimpleFoam` with Spalart–Allmaras on wall-resolved unstructured meshes at $\mathrm{Re} = 6{\cdot}10^6$ — a configuration close to ours. They report about 2× on the residual and, on the metric this paper also uses, **11× to reach 1% force error and 16× to reach 5%**. Their operator is a *region-to-point* map evaluated at a target point, so it is mesh-native by construction, and the criterion in §6 says a mesh-native seed retains the wall gradient. Their result is the largest near-wall warm-start gain in the literature and it is consistent with, not contrary to, what we measure.

Fuchi et al. [8], whose group also studies multi-fidelity learned flow models [22], report the largest number in this literature — **26.3× fewer iterations and 16.4× wall-clock** — from a convolutional wake-extension model. It is worth being precise about what that result contains, because at first reading it dwarfs everything here. Their method divides the domain into near-body, wake and off-body regions; the network predicts the *wake*, and the acceleration is reported to be achieved "when combined with an accurate flow prediction in the near-body region". The near-wall state is therefore supplied already correct, and the network's contribution is the region where no near-wall gradient exists to lose. §7.3 measures the complementary bound in our configuration: an **oracle** seed of the exact converged field across the entire downstream region is worth **+0.5%** on viscous drag here. The two results are about different regimes and compose rather than compete.

Sharpe et al. [7] initialise transient URANS with a point-based model combined with potential flow and report ~2×, using a drag-band metric close to ours; we measure `potentialFoam` alone as an arm and find it inert on drag (+0.6%). Hu et al. [13] predict at cell and wall-face centroids — mesh-native again — for a submarine hull and report 3.5× at a residual threshold of $5{\cdot}10^{-6}$, with cross-mesh generalisation. **Their validation is residual-based throughout and reports no force coefficient.** That is the common practice in this literature, and §5.6 is the reason we departed from it: on our configuration one seed is **+22.1% on the residual and −58.8% on total drag**. We are not claiming their result is wrong; we are reporting that on our cases the two metrics can disagree in sign, so we fixed a force metric before running the arms.

A third line puts the surrogate *inside* the solver rather than ahead of it: Sousa et al. [19] embed one in the PISO pressure-velocity loop and, importantly for §9, introduce a solver-intrinsic, hardware-

independent measure of effort that charges the surrogate's own overhead – the same accounting concern that makes us report iterations and seconds separately.

At the level of the linear algebra rather than the field, NOWS [4] supplies learned initial guesses to Krylov solvers and reports up to 90% time reduction; Oh et al. [5] constrain learned corrections so Newton convergence is preserved, reporting 5.4× at 6.4M DOF; Khodak et al. [6] select preconditioners online. These are complementary to and composable with an outer-field seed. Oh et al. are the closest prior art to our §6 in *spirit* — both say that L^2 accuracy is not what makes a seed good — and it is worth stating the difference plainly: theirs is a spectral property of the Jacobian, established for Newton solvers; ours is a named, measured, geometric defect of the *representation* (the first-cell wall gradient), with a closed form and a pre-flight test that needs no solver internals.

2.2 The near-wall region is independently known to be where these models fail

The mechanism we measure is not a surprise to the prediction community; what is new is its consequence for warm starting.

The AirfRANS benchmark paper [1] reports that models "have difficulties predicting wall shear stresses as velocity values at the closest nodes from the geometry are often largely overestimated", and identifies this as what damages the drag coefficient. §6 measures the same overestimate — a factor of ~20 — arising from the *representation alone*, with the exact converged field in place of any prediction.

The field audits itself in the same direction: benchmark evaluations of current architectures for aerodynamic prediction [20] and the ML4CFD competition retrospective [21] both report near-wall quantities as the weak point.

DD-RNO [14] argues that "a single neural architecture cannot simultaneously resolve sharp near-wall boundary layers and smooth far-field potential flow" and routes query points to separate inviscid, boundary-layer and wake decoders by wall distance. Zhang et al. [15] replace isotropic proximity modelling with explicit tangential–normal structure for the same reason. Both are motivated by prediction accuracy. That an independent line arrives at a *region split by wall distance* is convergent evidence for condition 2 of §5.3, reached from the other direction.

2.3 Mesh-native surrogates

Transolver [2], PCNO [3] and their successors [16, 17, 23] predict on native mesh points rather than on a raster, and the capability is presented as an accuracy or memory convenience. This paper's claim is that it is not a convenience: it is the difference between a seed that accelerates a solve and one that costs more than starting cold. A surrogate stored as a 128^2 image cannot be used for warm starting at all, and no amount of raster refinement recovers it (§7.1).

The point sharpens as the field moves to pretrained, general-purpose aerodynamic models [24], where the output representation is a design decision taken once and inherited by every downstream user. If such a model is ever to be handed to a solver, the criterion of §6 is a constraint on that decision.

2.4 Classical initialisation, which is what a practitioner actually uses

The comparator that matters is not a uniform freestream. Production aerodynamics warm starts by **grid sequencing**: solve on a coarsened mesh, map the result up, continue on the fine mesh — shipped in OpenFOAM as `mapFields`, and closely related to full-multigrid initialisation, which is

standard in structured compressible codes and still an active line in its own right [26]. A learned initialisation measured only against a cold start has not answered the question a practitioner asks.

We therefore run grid sequencing as an arm (§5.7), with the coarse solve charged in fine-mesh-equivalent iterations. It is also the criterion's out-of-sample test: a coarsened body-fitted mesh keeps its wall-normal stations clustered at the wall, so §6 **predicts** that grid sequencing preserves the first-cell gradient and helps — a prediction about a method containing no network, no training data and no surrogate, and one the criterion was not derived from.

2.5 Fallbacks and worst-case guarantees

Preserving worst-case behaviour by falling back to the classical method when a learned component is untrustworthy is an established pattern; Yavlovich et al. [9] apply it to linear assignment with a dual warm start and retain baseline runtime even at 100% fallback, and Schmidtobreick et al. [18] warm-start active-set solvers with a GNN while retaining convergence guarantees. **We cite this as prior art for the pattern.** What is ours is its instantiation for a PDE solver's initial *field*: a decision rule that reads only a short probe of the solve it is about to commit to, leave-one-case-out calibration, and a measured capture-versus-cost curve showing that longer probes are monotonically worse (§8). We note that Zhou et al. [12] report exactly the failure such a rule exists to catch — in their extrapolation case the initialisation gives no clear advantage and the residual sharply increases — and have no test for it.

3. Setup

Solver. OpenFOAM v2606 (ESI), `simpleFoam`, steady incompressible SIMPLEC (`consistent yes`), Spalart–Allmaras. `nNonOrthogonalCorrectors 2`. Under-relaxation `U` 0.7, `nuTilda` 0.4 — chosen, not defaulted; see §4 rule 2. Budget 6000 iterations. Every case is instrumented with both the `forceCoeffs` and `forces` function objects, so total, pressure and viscous drag are available separately at every iteration.

Mesh. Body-fitted C-grid generated by `blockMesh` from a specification we control (`solver/cgrid.py`): 31,700 cells, 20-chord far field, stitch-free wake cut via shared vertex ids, first cell 1e-5 chords tall (centre 5e-6), wall-normal grading to $y+ < 1$. The same mesh is used for every arm of every experiment, so mesh quality is never a differential effect.

Cases. NACA 4-digit sections at Re = 3e6, `u_inf = 1`, kinematic pressure. Two sets. The **mechanism study** is 5 cases (0012@4°, 2412@2°, 0015@6°, 0012@0°, 2415@5°) carrying fifteen seeding arms, which is where every controlled contrast and the acceptance gate are measured. The **generality corpus** is 13 cases (0012 at 8/10/12°, 2412 at 8/10°, 0018 at 4/8°, 4415 at 2/4°, 2415 at 2/8°, 0015 at 2/4°) carrying four arms, which is where the headline is measured. The two sets are disjoint, and every number states which it came from.

Surrogate. A Transolver-style point model, (B,N,7) -> (B,N,4), trained on AirfRANS under the conventions of the companion paper (dimensional fields, nu = 1.56e-5, Re from |u_in|). It is queried directly at the C-grid cell centres — `solver/surrogate_seed.py` — with no intermediate representation of any kind. Its training `sdf` distribution is centred on 0.23 chords, which is why seeds are cut off at 3.5 chords and why the outer field is left cold.

Arms. Every experiment carries a **converged-field oracle** as a control. The oracle is the cold run's own converged solution, re-injected as an initial condition, and it must post a large positive

saving before any other arm in that experiment is read. Across twelve experiments the control has read +68% to +99.9%. An experiment whose control fails is a broken measurement, and we say so rather than reporting its other arms.

4. How to measure a warm start without fooling yourself

This section is a contribution, not preamble. Each of the first six rules below changed a **sign** on this project's own data; rules 7 and 8 were added by the thirteen-case sweep and each cost us a result. All are implemented in `solver/scoring.py` or `scripts/reanalyse_depth.py`, each with a test that names the mistake it prevents.

1. **A residual threshold measures a convergence rate only while the residual is still falling.** Report every threshold as a multiple of the run's residual floor and refuse to read anything below ~5x. The same arm read +15%, +31% and +13% at 1.9x, 1.3x and 0.9x the floor — noise presented as a trend.
2. **The floor itself is usually an artifact, so find out which one.** Ours was under-relaxation at 0.9 with SIMPLEC, not the 2e5-aspect-ratio cells: `U` 0.7 / `nuTilda` 0.4 moved the floor 30x, from 1.1e-5 to 3.5e-7. Ten other variants — longer budgets, tighter inner tolerances, a better wake mesh — bought nothing.
3. **An arm that never reaches the target must be bounded at its full budget, not dropped.** Dropping non-finishers turned a true -199.4% into -31.2%: the failing arm was rewarded for failing.
4. **Score every arm against one external reference**, never against its own final value. Grading the oracle against itself turned a +73.5% control into +1.0%.
5. **Only arms whose coefficient has stopped moving may define that reference.** Peak-to-peak over the last tenth of the run, against a quarter of the band. A single diverged arm previously set the reference and condemned an entire table; restricting the reference to settled arms moved the spread from 3.104% to 0.334%.
6. **`nNonOrthogonalCorrectors` multiplies the pressure history.** Parse per `Time` block; zipping fields by index across a log made pressure lag velocity 3:1 and moved every shallow number.

One consequence of rule 5 is worth stating even though it is small here. Because the reference is a median over the arms that have settled, *adding* an arm to a tree re-scores every arm already in it. **A number must therefore name the arm set it was computed over**, and ours does (`-drop-arm` declares it). We measured the sensitivity rather than assuming it away: removing our most recently added arm moves every headline number by at most 0.5 percentage points.

The primary metric. `iterations_to_force_band`: the first iteration after which a coefficient stays within +/-b of the reference for the rest of the run. Saving is 1 - warm/cold, bounded at the budget. We report b = 1%, 0.5%, 0.2% and state which bands are readable.

Readability. A band is readable only if the settled arms agree about the converged value to well inside it (rule 5) — concretely, to within half the band. On the core study the largest disagreement is 0.232% of Cd, so b = 1% and b = 0.5% are readable (limits 0.5% and 0.25%) and b = 0.2% is not (limit 0.1%). **The 0.5% verdict is thin**: 0.232% against a 0.25% limit. We report the margin rather than only the verdict, because one additional case can flip it.

Provisional. Every readability verdict in this paper is a property of the case set, since the reference is a median over cases and arms. The 13-case sweep recomputes all of them, and they are re-checked

rather than inherited.

The saving depends on the band, and the two drag components differ. On the five-case mechanism study:

arm	Cd@1%	Cd@0.5%	Cd@0.2%	Cd_v@1%	Cd_v@0.5%	Cd_v@0.2%
`nf_bl`	**+33.9%**	**-4.2%**	-38.5% *(unreadable)*	**+14.6%**	**+13.7%**	+11.0% *(unreadable)*
`oracle_mesh`	+92.1%	+92.4%	+93.1%	+92.5%	+91.9%	+91.3%

Viscous drag is monotone across bands and total drag is not. Monotone stability *is* the evidence that a number is a convergence-rate measurement rather than an artifact of where a wandering curve happens to cross a line — and `Cd_v` is 60–84% of the drag here.

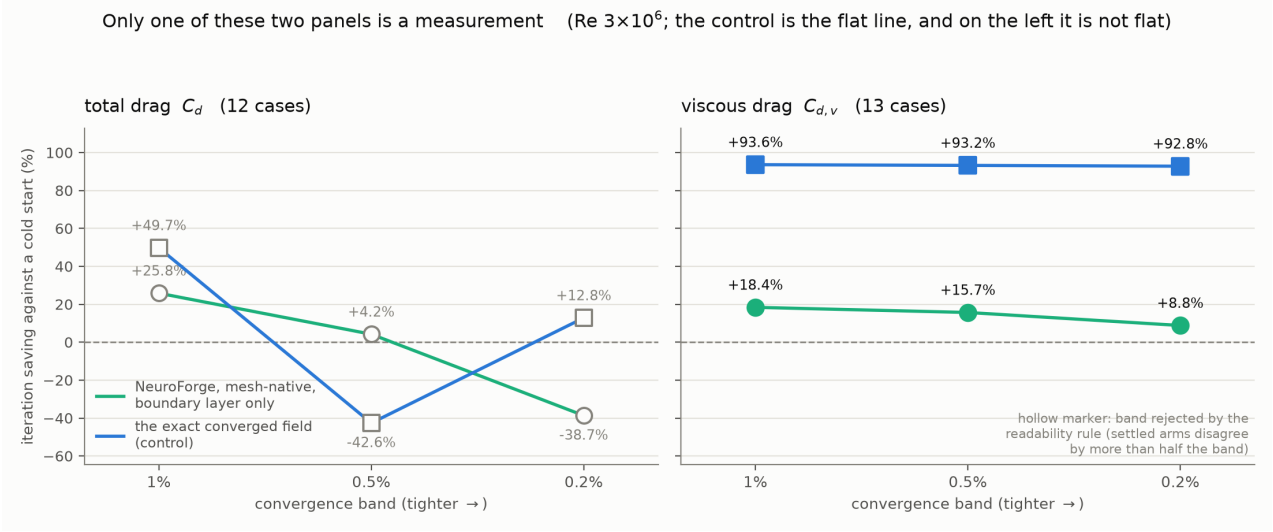

Figure 2. Only one of these two quantities is a convergence-rate measurement. Iteration saving against a cold start as the convergence band is tightened, on the thirteen-case corpus. On **total drag** (left) every band is rejected by the readability rule, *and the converged-field control itself swings +49.7% → −42.6% → +12.8%* — a control that is not flat indicates a measurement that cannot be read. On **viscous drag** (right) the control is flat at +93% and the trained seed is monotone at +18.4% / +15.7% / +8.8%. Hollow markers are bands the readability rule rejected; they are plotted rather than deleted, because a curve with its last point removed would look steadier than the measurement is.

A withdrawal. A previously recorded +41.8% at Cd@0.5% was read against a reference that a diverged arm had moved. On the settled reference over the declared arm set that row is **-4.2%** — readable, and negative. The +33.9% at Cd@1% is unaffected on the five-case study, and §5.1 reports what happens to it at thirteen. We report this because a protocol that only ever deleted inconvenient numbers would not be a protocol.

Rule 7, learned from the corpus: an admission threshold and a scoring threshold must be derived from the same quantity. We pre-registered a gate that admits a case if its arms agree on final drag to within 2%, and a scoring rule that can read a 1% band only if they agree to

within 0.5%. Both were declared in advance, and they disagree — so the sweep admitted two cases it could not then read, and they cost us every total-drag row (§5.1). The gate should have been derived from the readability limit. We did not change it after the fact; we report the inconsistency, because a study can admit cases it cannot score and the failure is invisible until it happens.

Rule 8, from the same sweep: a converged run is finished, not truncated. OpenFOAM exits early when `residualControl` is satisfied. A scorer that judges completeness by run length alone calls that a truncation and discards it — penalising precisely the arms that converge *fastest*. Ours did, and it silently dropped the oracle control on one case, turning a +93% control into a +49.7% bound.

Statistics. Savings 1 - warm/cold are left-skewed, so we report percentile bootstrap 95% CIs (10,000 resamples) rather than t-intervals, plus an exact two-sided sign test. At $n = 5$ the smallest attainable p is 0.0625; we state this wherever $n < 6$ rather than reporting a non-significant p as if the test could have succeeded. The thirteen-case corpus of §5.1 is where the sign test can bite: 13/13 gives $p = 0.0002$.

5. The result, and the three conditions behind it

Two studies, disjoint (§3). The **corpus** — thirteen cases, four arms — carries the headline and answers *does this generalise*. The **mechanism study** — five cases, fifteen arms — carries the controlled contrasts and answers *why, and under what conditions*. §5.1 is the first; §5.2–§5.4 are the second.

5.1 The headline, at thirteen cases

`scripts/corpus_probe.py`. Thirteen cases — five NACA sections, 2° to 12°, from attached flow into incipient separation — each admitted under a gate declared before the sweep ran (residual floor $\leq$ 1e-5, arms agreeing on final Cd to within 2%). **All thirteen were admitted**, so nothing here rests on a discretionary exclusion.

row	`nf_bl`	95% CI	wins	sign test	oracle control	Cartesian control
Cd_v@1%	**+18.4%**	[+12.4, +25.3]	**13/13**	**p = 0.0002**	+93.6% (13/13)	+3.4% (p = 0.27)
Cd_v@0.5%	+15.7%	[+10.6, +21.8]	13/13	p = 0.0002	+93.2%	+4.9% (p = 0.27)
Cd_v@0.2%	+8.8%	[+4.3, +13.5]	11/13	p = 0.022	+92.8%	−43.6% (p = 0.58)

Per case at the 1% band: **+3, +9, +9, +10, +11, +11, +16, +17, +19, +25, +28, +35, +47**. There is no losing case and no case carrying the mean.

Three properties make this a rate measurement rather than a crossing artifact: it is **monotone** across every band, the **oracle control** is flat at +93% and wins 13/13, and the **negative control** — the exact converged field resampled onto a 128^2 Cartesian grid — is statistically indistinguishable from no seed at all (+3.4%, p = 0.27). A pipeline that manufactured savings would not produce a null there.

Total drag, lift and pressure drag are unreadable over the thirteen, and two cases carry all of it. `naca4415` at 2° and 4° have arms that settle on drag values **1.04% and 1.27% apart**, against ≤0.113% for every other case in the corpus. No arm is *unsettled* on those two — they settle in different places, which is the signature of a case without a unique steady fixed point, and the same

signature `naca4412@3` showed. All three are thick cambered sections at low incidence, and all three have the corpus's worst residual floors (8.75e-6 and 1.22e-6, against ~1e-7 elsewhere).

Sensitivity analysis, and it is post hoc. Dropping those two cases leaves eleven, on which Cd@1% becomes readable and reads **+30.0% [+5.3, +47.0], 10/11 wins, p = 0.012**, with the oracle control at +95.4% (11/11) and the Cartesian control at −262.5%. We report it because withholding it would be hiding a result, and we label it because the exclusion was chosen after seeing the data. It is not the headline. We do **not** carry the Cd@0.5% row from that subset: the oracle control there reads −13.9% with one case at −1112%, so by our own rule the row is unreadable regardless of what `nf_bl` does.

The size of the effect, stated plainly. +18.4% is a modest acceleration, and we do not present it as more than that. What the thirteen cases establish is not a large saving but a *reliable* one: every case positive, a converged-field control at +93.6%, a negative control indistinguishable from no seed, and a monotone response across convergence bands. On a quantity that is 60–84% of the drag, obtained from a trained model on airfoils and incidences it had not seen.

The three conditions, and the study that isolates them

Everything from here to §5.6 is the **mechanism study**: `scripts/mesh_native_probe.py`, five cases, **one prediction**, one variable changed per arm, all twenty solves complete. Scored over the thirteen-arm `repr3` set with `oracle_wake` dropped (§4); including it moves no entry below by more than 0.5 points.

arm	what it hands over	residual 5e-6	**Cd@1%**	Cl@1%	Cd_v@1%
`nf_bl`	u, v, nut in the BL, mesh-native	< -80.5%	**+33.9%**	+10.1%	+14.6%
`nf_bl_nut`	eddy viscosity only	+1.2%	**-293.2%**	**+41.1%**	**+42.4%**
`nf_bl_vel`	velocity only	-8.2%	-40.3%	-10.3%	-4.9%

The recommended arm in this table, `nf_bl`, is the one measured at thirteen cases in §5.1. Its Cd@1% here is +33.9%, and +34.1% when re-measured in the independent tree of §5.2; §5.1 is where that number meets a corpus.

The resampled arm is not in this table, and the reason is a correction. This study originally carried a `nf_bl_proj` arm — the same prediction through a 256×64 round trip — reported at −58.8% on `C_d`@1%. That round trip took its wall-normal coordinate from the nearest surface *vertex* rather than the nearest segment, which reads this mesh's first cell ring three orders of magnitude too far from the wall (§5.2.1). Re-measured with that fixed, the same arm reads −32.1% with a 95% interval of [−144.1, +60.9] and 3 wins in 5 — **not distinguishable from zero**. The representation contrast is therefore made in §5.2 on the *exact converged field*, where it is clean, rather than on this network prediction, where it is not.

5.2 Condition 1 — the surrogate must be evaluated at the solver's own points

The controlled contrast is one field handed to the solver through different representations, with everything else held fixed. We make it on the **exact converged field**, so that no property of any

network enters: the arms differ only in how that field was stored and read back.

arm (exact converged field)	representation	C_d@1%	95% CI	wins
oracle_mesh	the solver's own cell centres	**+92.0%**		5/5
or_proj_coarse	wall-fitted 256×64 from $2.5{\cdot}10^{-4}$	**−181.6%**	[−318.0, −45.2]	1/5
or_proj_fine	wall-fitted 256×64 from $5{\cdot}10^{-6}$	**−266.8%**	[−419.5, −114.1]	0/5
or_proj_half	wall-fitted 256×32 from $5{\cdot}10^{-6}$	−183.3%	[−333.0, −33.7]	2/5
cartesian_128	uniform 128^2 raster	**−548.4%**		

A 274-point swing on one field, from representation alone, and the intervals on the projected arms exclude zero. The same converged solution that saves 92% of a cold solve when the solver reads it at its own cell centres costs the solver two to five times a cold start when it is stored on a grid first.

On the network arms this contrast is not statistically resolvable, and we say so. nf_bl reads +34.1% [+14.1, +52.6], 4/5, and nf_proj_coarse reads −32.1% [−144.1, +60.9], 3/5 — an interval spanning zero, with the mean set by a single case at −231%. The network's own field is not accurate enough for the representation damage to be visible against it. The claim is made on the oracle arms, where it is clean, and the network arms are reported for completeness.

*5.2.1 It is **not** the near-wall state — a result we did not expect*

The obvious explanation is that a grid cannot hold the near-wall state. §6 shows that damage is real, computable and bounded. **It is not what costs the solve**, and the placement ladder shows it directly. All three projected arms below carry the same exact field through the same round trip; only the grading of the grid changes.

arm	values	first station	wall-gradient error	roughness (× conv.)	C_d@1%	C_d,v@1%
or_proj_coarse	16,384	$2.5{\cdot}10^{-4}$	**1218.8%**	16.8×	**−181.6%**	**+86.1%**
or_proj_fine	16,384	$5{\cdot}10^{-6}$	**1.8%**	**1.0**×	**−266.8%**	+69.1%
or_proj_half	8,192	$5{\cdot}10^{-6}$	**1.8%**	**1.0**×	−183.3%	+71.0%
oracle_mesh	—	native	0.0%	1.0×	+92.0%	+92.5%

Two things follow, and the second is the important one.

Placement, not budget, determines what a representation keeps. Moving the first station inside the mesh's first cell takes the wall-gradient error from 1218.8% to **1.8%** and the roughness to exactly the converged field's, and it does so at **half** the value budget as readily as at the full one. That is §6's criterion, confirmed: what a grid retains is set by where it samples, not by how much it stores.

And it does not help. It hurts. The arm that reproduces the near-wall state to 1.8% and its roughness exactly converges at **−266.8%, winning 0 of 5 cases** — *worse* than the arm carrying 1218.8% gradient error at −181.6%. A seed can be a near-perfect reproduction of the converged boundary layer and be among the worst initial conditions in this study.

So the near-wall velocity field is not the mediator of the harm. §5.5 and §6.7 close off the two remaining ways it might have been, and §5.2.2 reports where the damage actually is.

5.2.2 The damage is to the pressure field

The `C_d,v` column above is the tell. **Every projection preserves viscous drag**: the oracle through a grid reads +86.1% [+84.2, +87.9], +69.1% and +71.0%, all 5/5, against the mesh-native oracle's +92.5%. Viscous drag is 60–84% of the drag here and it is the quantity the near-wall gradient integrates — and it survives the round trip essentially intact even when the gradient is 1219% wrong.

What does not survive is total drag, and therefore the pressure component: `or_proj` reads **−183.9%** on `C_d,p`@0.5%. Pressure drag is 16–40% of the drag, converges three times slower than total drag from cold, and is the channel §5.4 identifies as the one an inconsistent seed destroys.

This is the mechanism the paper can support: **a projection preserves the near-wall velocity well enough for wall shear and corrupts the pressure field.** It explains why restoring the wall gradient — by placement (§5.2.1), by wall-law repair, or by smoothing that repair (§5.5) — never recovers the solve, and why the only arm in the study that is *positive* on pressure drag is the one that smoothed the reconstructed wall shear (§5.5).

5.3 Condition 2 — hand over the boundary layer only

The region axis is controlled the same way as the resampling axis: both arms are the same network, both mesh-native, differing only in whether the handover is masked to the boundary layer. Cd@1%, five cases, cold = 805 iterations, oracle control **+92.1%**:

arm (network, mesh-native)	region handed over	Cd@1%
`nf_bl`	boundary layer only	**+33.9%**
`nf_mesh`	the whole field	**< -568.3%**

Why it fails is not what we first wrote, and the correction is measured. The natural explanation is that the model extrapolates in the outer field — its training `sdf` distribution is centred on 0.23 chords while the C-grid reaches 20. That explanation is **falsified by §5.7**: grid sequencing hands over a *coarse mesh solution*, which extrapolates nowhere, and the boundary-layer-restricted arm still beats the whole-field one on every readable row (+75.9% against +63.1% on `C_d,v`, −302.4% against −375.4% on `C_d`). So the restriction is not about the surrogate's trust region.

The next explanation — that a mapped whole field is not divergence-free on the fine mesh while a freestream outer field is — is falsified too, from the solver's own first continuity error: every seed but the converged-field oracle sits at ~$1.4{\cdot}10^{-6}$, and the whole-field arm is if anything the *cleanest* of them at $1.19{\cdot}10^{-6}$ while performing worst. Divergence does not discriminate.

What survives is the consistency mechanism of §5.4: a seeded region that is inconsistent with the state the solver holds elsewhere destroys the pressure field, and the larger that region the more there is to be inconsistent about. We state this as the surviving candidate, not as an established mechanism.

Together with §5.2 this gives two controlled contrasts sharing a common arm (`nf_bl`), one per axis, each changing a single variable. Neither factor alone is sufficient: mesh-native evaluation of the whole field is the worst arm in the study, and boundary-layer restriction under resampling is

still negative.

We deliberately do **not** present these as a 2×2. The fourth corner — a network prediction of the whole field, resampled — was never run, and the arms that would fill it come from a different population (oracle rather than network). A table whose cells mix provenance would read as a design when it is a gap.

The oracle bound on the resampled row, which removes the accuracy confound entirely, is reported separately and is the stronger statement:

arm	field	representation	Cd@1%
`fitted_256x64`	exact converged answer	wall-fitted 256x64	-172.6%
`fitted_bl`	exact converged answer, BL only	wall-fitted 256x64	-206.1%
`cartesian_128`	exact converged answer	Cartesian 128^2	-548.4%

Even a perfect field, resampled, costs the solver more than starting it cold.

5.4 Condition 3 — velocity and eddy viscosity together

This condition was not anticipated; it came out of splitting the channels.

Eddy viscosity alone is simultaneously the **best** arm in the study on viscous drag (+42.4%) and lift (+41.1%), matching the oracle projection, and the **worst** on total drag (-293.2%). Velocity alone is bad at everything. Only the pair is positive on total drag.

The reason is the structure of the Spalart–Allmaras production term, which is driven by the strain rate. An eddy viscosity handed over without the velocity field that generated it is inconsistent with the strain the solver computes from the field it actually has; the momentum sink is therefore wrong, the pressure field must reorganise, and `Cd_p` — 16–40% of the drag here — is destroyed. The shear-driven quantities do not care, and speed up.

Consistency between handed-over channels is not a detail of the recipe. It is most of it, and §7.3 finds the same lesson again at a completely different length scale.

5.5 Restoring the wall gradient does not recover the solve

Section 6 shows the damage a projection does is available in closed form. A factor that is known can be divided back out, so we built the repair the mechanism implies: invert the law of the wall at the representation's own first station to recover `u_tau`, using **only what the representation already carries**, and re-evaluate the profile at each cell's own wall distance (`solver/placement.py`).

It works on the gradient, and then on its smoothness too. The first repair restored the magnitude but left the reconstruction rough along the surface, because every station is inverted independently from its own value. Smoothing the recovered `u_tau` over an arclength window fixes that. Measured on the seeds exactly as the solver received them, five cases:

arm	first-cell gradient error	roughness (× converged)
`nf_bl` (mesh-native, the seed that works)	53.7%	4.2×
`nf_proj`	877.7%	20.6×
`nf_proj_fix` (repaired)	**69.4%**	26.2×
`nf_proj_smooth` (repaired + smoothed)	**73.0%**	**5.7**×
`or_proj`	1218.8%	16.8×
`or_proj_fix`	54.4%	21.5×

`nf_proj_smooth` matches the working seed on **both** diagnostics this study can measure: 73.0% gradient error against its 53.7%, and 5.7× roughness against its 4.2×, where the unrepaired projection is at 877.7% and 20.6×.

And the solve does not care.

arm	`C_d`@1%	`C_d,v`@1%
`nf_bl` (mesh-native)	**+34.3%**	+14.6%
`nf_proj`	−32.1%	+14.5%
`nf_proj_fix`	−32.0%	+11.7%
`nf_proj_smooth`	−31.4%	+11.5%
`or_proj`	−182.1%	+86.1%
`or_proj_fix`	−180.7%	+56.7%

Three seeds spanning **877.7% to 69.4%** in gradient error and **26.2× to 5.7×** in roughness all land between −31.4% and −32.1% on total drag, where the mesh-native seed reads +34.3%. On viscous drag the repair is not merely neutral but slightly harmful. And `nf_proj`, carrying sixteen times the mesh-native seed's gradient error, already matches it on viscous drag (+14.5% against +14.6%).

We report this as a falsification, because that is what it is. We predicted the repair would work, registered the reasoning before running it (`docs/protocols/placement_prediction.md`), built it, found the one measurable difference that remained, removed that too, and it still did not work. §6.7 states what that costs the paper's mechanism, which is a great deal.

One secondary observation, flagged rather than promoted. On *pressure* drag — which converges three times slower than total drag here and whose 1% row is unreadable — the smoothed repair is the only seed in this study that is positive: **+19.8%** at the 0.5% band and **+57.8%** at 0.2%, against −115.4% for the recommended seed. A rough wall-shear distribution driving a spurious pressure response is a coherent reading, and it fits §5.4's finding that pressure is where inconsistent seeds do their damage. It was found after the fact, on a secondary quantity, so §4's protocol says it is an observation and not a result. The experiment that would settle it is a smoothed *mesh-native* seed, which this study does not contain.

5.6 The residual objection

`nf_bl` is **negative on the residual at every depth** and positive on drag; the projected arms are the exact inverse. A reviewer is right to look hard at that. Our answer has three parts.

1. **The residual is not the objective.** Nobody runs a RANS solve to obtain a small residual; they run it to obtain a force coefficient that has stopped moving. The residual is a proxy, and

this study measures the proxy failing — on the *exact converged field*, so the disagreement cannot be blamed on our network:

arm (exact field)	residual $5{\cdot}10^{-6}$	`C_d`@1%	wins
`or_proj_coarse`	**+40.6%**	**−181.6%** [−318, −45]	1/5
`or_proj_fine`	**+36.6%**	**−266.8%** [−419, −114]	0/5
`nf_bl` (recommended)	**−80.5%**	**+34.1%** [+14, +53]	4/5

The two arms that look best on the residual are the two worst on drag, and the arm we recommend is the worst on the residual. **A study that scored the residual alone would have selected exactly the wrong seed**, and would have done so with intervals excluding zero. We report both metrics, always.

1. **We know what the residual is rewarding.** The `Ux` residual measures how much the field changes per iteration, so it rewards smoothness. A resampled field has had its near-wall structure interpolated away and therefore changes less, while carrying 877.7% error in the wall gradient. The residual is not being fooled at random — it is faithfully measuring something that is not drag.
2. **The choice of metric was pre-committed and cuts against us.** The force metric replaced the residual metric for the reason in §4 rule 1, which predates this arm; and the readability rule then rejected rows we would rather have quoted, including the withdrawal above.

The honest residue stays in the paper: **on the residual, `nf_bl` is worse than a cold start.** §8 is what makes that survivable — the same 25-iteration gate that bounds drag bounds the residual metric's worst case at -7.6%.

5.7 The classical baseline: grid sequencing

The comparator that matters is not a uniform freestream. Production aerodynamics warm starts by **grid sequencing** — solve on a coarsened mesh, map the result up, continue on the fine one — and a learned initialisation measured only against a cold start has not answered the question a practitioner asks. We run it as an arm: the same C-grid family coarsened by two (7,850 cells against 31,700, first cell $2{\cdot}10^{-5}$ against 10^{-5}), mapped with a nearest-cell map in body-fitted coordinates, and its coarse solve **charged**.

row	cold	oracle	`nf_bl`	`sequenced_bl`	`sequenced_vel`	`sequenced_nut`	`sequenced`
Cd@1%	802	+92.1%	+33.9%	−302.4%	−4.2%	—	−375.4%
Cd_v@1%	696	+92.4%	+14.6%	**+75.9%**	+2.2%	+34.9%	+63.1%
Cl@1%	944	+99.9%	+10.1%	+49.4%	−0.1%	−59.7%	+33.1%

Three things follow, and none of them is "ours is better".

The classical seed is the better seed. On viscous drag grid sequencing reads +75.9% against our +14.6% — five times the saving. That is exactly what §6 predicts for it: a coarsened body-fitted mesh keeps its stations clustered at the wall, so it satisfies condition 1 by construction. **The criterion correctly predicts the behaviour of a method that contains no network, no training data and no surrogate, and from which it was not derived.**

And it still is not worth running here. The coarse solve is a real cost. Converted at the cell-count ratio — the conservative direction, since a coarse cell is cheaper — it charges **1486 fine-equivalent iterations against a cold run of 696**. The saving cannot pay for that. Our seed's advantage is therefore not quality but **price**: ~11 s of inference against a second solve. That is the honest comparison, and it is a different claim from the one the warm-start literature usually makes.

The channel condition holds for it too, and total drag is destroyed. The split mirrors §5.4's exactly: the saving rides on the eddy viscosity (+34.9% alone) while velocity alone is inert (+2.2%). Unlike `nf_bl`, however, handing both channels over does **not** rescue total drag (−302.4%). The consistency condition of §5.4 is evidently about consistency *at the fine mesh's resolution*, which a coarse velocity field does not have — it carries a `nut` generated by a strain field the fine mesh does not reproduce.

Caveats, stated rather than buried. The coarse solve ran its full 6000 iterations, so the charge above is pessimistic; a practitioner would stop it earlier. And the mapper is ours, not a production `mapFields`: it leaves a first-cell gradient overestimate of order 6–10× where the coarse mesh's placement alone would permit about 2×, so this arm is a **lower bound** on what grid sequencing can do. Both caveats point the same way — grid sequencing would look better, not worse, with more care — and neither changes the conclusion that its seed is good and its price is a second solve.

5.8 A common-mode limitation, checked rather than assumed

`write_case` floors `nuTilda` at its freestream value, which clips 37% of the boundary-layer cells. That sounds like it could produce these swings. It cannot: the clipped cells hold values ~88x below the peak, so the floor removes only **2.1–2.3%** of the eddy-viscosity field's energy, and it applies identically to every arm including the oracle. It is a common-mode limitation of the study, not a differential effect that could move an arm from +33.9% to -293.2%.

6. The mechanism, in closed form

Everything in §5 follows from one quantity, and that quantity can be computed before any solve is run, from two numbers a CFD engineer already has.

6.1 What a projection actually does to the near-wall state

Viscous drag is the surface integral of the tangential velocity gradient in the first cell off the wall. On the meshes this paper uses that cell centre sits $5{\cdot}10^{-6}$ chords from the surface, and viscous drag is 60–84% of the total.

When a field is resampled through a grid, every mesh cell nearer the wall than the grid's first wall-normal station receives the value belonging to that station. There is nothing else for it to receive: the representation holds no sample in between. The reconstructed first-cell gradient is therefore not *degraded* — it is replaced by a different quantity, the velocity at the station divided by the distance to the cell.

Write `h1` for the representation's first wall-normal station, `y_c` for the mesh's first cell centre, and `u(y)` for the true near-wall profile. The seeded first-cell gradient is `u(h1)/y_c` where the true one is `u(y_c)/y_c`, so the gradient is overestimated by

$$\mathbf{G = u(h_1) \;/\; u(y\_c)}$$

and in wall units this is a statement about the law of the wall alone:

$$\mathbf{G = u^+(y_1{}^+) \;/\; u^+(y\_c^+)}, \texttt{ y+ = y u\_tau / nu}$$

with `u+ = y+` in the viscous sublayer and `u+ = ln(y+)/kappa + B` in the log layer. **There is no fitted parameter in this expression.** `u_tau` comes from the converged solution's own wall gradient and `nu` from the case.

6.2 It is a parameter-free upper bound on the damage

Table 5 tests the closed form against the measured first-cell gradient of a wall-fitted 256×64 projection of the **exact converged field**, over five cases, at five first-station heights spanning a factor of fifty. `u_tau` is taken from each case's own converged wall gradient; nothing is tuned.

first station	`y+`	predicted `G`	measured `G`	predicted/measured
$2.5{\cdot}10^{-4}$	38	23.3×	14.4×	1.63 ± 0.09
$1.0{\cdot}10^{-4}$	15	16.9×	7.8×	2.17 ± 0.09
$2.5{\cdot}10^{-5}$	3.8	6.25×	2.50×	2.55 ± 0.33
$1.0{\cdot}10^{-5}$	1.5	2.50×	1.00×	2.50
$5.0{\cdot}10^{-6}$	0.8	1.25×	1.00×	1.25

The expression over-predicts in every row, by between 1.3× and 2.6×. We report it as what it is: a *parameter-free upper bound*, correct in direction and in ordering, never optimistic. It is not a 13%-accurate estimate, and an earlier version of this work claimed that it was — that figure came from a projection whose wall-normal coordinate was measured to the nearest surface *vertex* rather than the nearest segment, mis-placing every near-wall cell by three orders of magnitude. The bug is fixed, the number is corrected, and the correction is recorded here rather than quietly absorbed.

Why it over-predicts is visible in the last two rows. The implementation clips a query below its first station rather than extrapolating, so once `y1+` approaches the mesh's own first cell the grid's first row is populated *from* that cell and the damage collapses to 1.00× — sooner than an idealised resampling would. A bound is the honest reading of an expression that assumes the worst about an implementation detail.

What the bound is for. Ruling a format out. At `y+ = 38` it says "expect order 20×", and 14.4× is measured; at `y+ < 1` it says "expect nothing", and nothing is what happens. Between those, it orders representations correctly without a solve, which is the decision it exists to support.

6.3 Two consequences that decide how a surrogate should be built

Refining the raster cannot fix this, and the formula says why. `u+` grows *logarithmically* in `y+`. Going from a 128^2 to a 512^2 raster spends sixteen times the values and moves `h1` by a factor of four, which moves `u+(y1+)` by `ln(4)/kappa ~ 3.4` on a value of ~23 — about 15%. That is the

closed-form version of the measured resolution ladder in §7.1, which is flat from 128^2 to 421^2, and of the estimate that one cell across the inner layer would need N ≈ 11,800, some 28× beyond what the standard datasets hold.

Placement, not budget, is what a representation's retention depends on. The alternative to sixteen times the values is to move the first station inside the first cell. A 64-level geometric stack from $5{\cdot}10^{-6}$ to 1 chord has a growth ratio of 1.214; a 32-level stack has 1.483. Both are ordinary meshes, and the second holds *half* the values of the 256×64 grid that loses the gradient. §5.2.1 confirms this directly: the move takes the measured gradient error from 1218.8% to 1.8% and does it as readily at 8,192 values as at 16,384.

This is a statement about retention, and only about retention. §5.2.1 also shows that the correctly-graded grid **converges worse** than the badly-graded one. Getting the near-wall state right is not what makes a warm start work, and nothing in §6 should be read as advice to grade a surrogate's output for speed. What §6 buys is a cheap, conservative way to know what a format keeps — which is worth having, and is not the same thing.

6.4 The pre-flight check

The criterion is therefore executable. Given a target mesh's first cell height and the wall-normal shape of the format a surrogate would emit, `G` follows immediately, and with it a verdict: a representation with no station inside the first cell has no sample of the state that viscous drag integrates, and will misreport it by roughly `G`. We ship this as `neuroforge.solver.placement` and as a command-line tool, so that the check can be run on a mesh and a format that have nothing to do with this study:

“ `python scripts/preflight.py -first-cell 1e-5 -re 3e6 -fitted 256x64@2.5e-4 ... predicted wall-gradient overestimate 19.18x [wall_law] FAILS. Expect the first-cell wall gradient to be overestimated by about 19.2x.` “

The criterion is necessary, not sufficient, and the paper is careful about this. Every representation that loses the gradient costs the solve, without exception across every arm measured here — so the check rules formats *out* for free. It does not rule them in, and we have two independent demonstrations of that. `nf_mesh` retains the gradient perfectly and is the worst arm in the study. And §5.5 restores the gradient of a failing representation to *better than mesh-native* and the solve still does not recover. Conditions 2 and 3 exist for this reason, and the pre-flight check should be read as a veto, never as an endorsement.

6.5 The criterion applied to the formats the field actually ships

Table 6 evaluates the closed form for the output formats a surrogate might emit, against the mesh used throughout this paper (first cell 10^{-5} chords, `u_tau` = 0.0477, `nu` = $3.33{\cdot}10^{-7}$). It costs no solve and no network, and it is the whole argument in one place.

output format	values	first station	y+	predicted G	verdict
uniform raster 128^2, 3-chord crop	16,384	$1.2{\cdot}10^{-2}$	1700	29.3×	fails (bound)
uniform raster 256^2, 3-chord crop	65,536	$5.9{\cdot}10^{-3}$	840	29.3×	fails (bound)
uniform raster 512^2, 3-chord crop	**262,144**	$2.9{\cdot}10^{-3}$	420	**27.6×**	**fails**
uniform raster 128^2, 1-chord crop	16,384	$3.9{\cdot}10^{-3}$	560	28.6×	fails
wall-fitted 256×64 from $2.5{\cdot}10^{-4}$	16,384	$2.5{\cdot}10^{-4}$	36	19.2×	fails
wall-fitted 256×64 from $2.5{\cdot}10^{-5}$	16,384	$2.5{\cdot}10^{-5}$	3.6	5.0×	fails
wall-fitted 256×64 from $5{\cdot}10^{-6}$	16,384	$5.0{\cdot}10^{-6}$	0.72	1.0×	**passes**
wall-fitted 256×32 from $5{\cdot}10^{-6}$	**8,192**	$5.0{\cdot}10^{-6}$	0.72	**1.0×**	**passes**
mesh-native, queried at cell centres	native	$5.0{\cdot}10^{-6}$	0.72	1.0×	**passes**

Two rows carry the paper. A **512^2 raster holds 262,144 values and still fails**, at 27.6×; a **wall-fitted grid of 8,192 values — one thirty-second of that budget — passes**. Sixteen times the values cannot buy what one grading decision gives away for free.

The practical reading is a design rule, not a ranking of architectures. Any surrogate whose output is a uniform raster over a crop of order the chord cannot warm-start a wall-resolved RANS mesh, however finely it is rasterised, because a uniform grid must resolve its smallest scale everywhere and the near-wall scale collapses like `nu/u_tau`. Surrogates that predict on native mesh points satisfy the criterion by construction. Between the two sits a large and mostly unexplored middle — graded wall-fitted outputs — where the criterion is satisfied or not purely by the choice of first station. §5.2 measures that middle directly, and §5.5 shows that reconstructing the missing near-wall profile after the fact is **not** an adequate substitute for placing a station there in the first place.

6.6 The criterion across Reynolds number, at no compute cost

Because the closed form is written in wall units it makes a prediction about a regime this study never set out to measure, and it is not the obvious one:

On the same mesh, a lower Reynolds number makes the projection worse.

Both `y+` values fall together as `nu` rises, but through different parts of the profile. The mesh's first cell sinks deeper into the *linear* sublayer, where `u+ = y+` falls in proportion; the representation's fixed station at $2.5{\cdot}10^{-4}$ remains in the buffer or log layer, where `u+` falls only logarithmically. The ratio — the damage — therefore grows.

This is testable with no new solves. Converged cold solves on the *same* C-grid already exist at Re = 10^3 to $3{\cdot}10^6$. Projecting each through the same wall-fitted grid gives Table 7 (`scripts/reynolds_transfer.py`):

Re	y+ first cell	y+ station	predicted G	measured G	pred/meas	weak-shear surface
10^3	0.001	0.1	62.5×	21.7×	2.88	61%
10^4	0.004	0.3	62.5×	21.6×	2.90	62%
10^5	0.027	1.7	62.5×	23.2×	2.69	42%
10^6	0.21	13.3	44.9×	20.8×	2.16	11%
$3{\cdot}10^6$	0.58	36.1	**23.8**×	**15.0**×	**1.62**	6%

The direction holds across three and a half decades: the same representation on the same mesh costs 15.0× at $\mathrm{Re} = 3{\cdot}10^6$ and 21.7× at $\mathrm{Re} = 10^3$. A practitioner's instinct — that a coarse representation is more forgiving at low Reynolds number, where the flow is smoother — is the wrong way round, and the reason is that the mesh's first cell has moved into the linear sublayer while the representation's has not.

The effect is real but modest, and it saturates. The measured damage rises by about 45% over that range rather than by the factor of three the unbounded expression suggests, and it is flat below $\mathrm{Re} = 10^5$. The bound loosens in the same direction: predicted/measured grows from 1.6 at $\mathrm{Re} = 3{\cdot}10^6$ to 2.9 at $\mathrm{Re} = 10^3$. The last column says why. It reports the fraction of surface stations carrying under a tenth of the peak wall shear — the signature of a laminar or separated layer. At $\mathrm{Re} \geq 10^6$ it is 6–11%; at $\mathrm{Re} \leq 10^5$ it is 42–62%, the boundary layer is laminar and largely separated, and the law of the wall does not describe it at all. The expression is then a bound in the loosest sense: it still gets the sign and the ordering right, and it still errs conservatively.

These numbers were re-measured on 2026-09-01 after the wall-distance defect described in §5.2 was fixed. The earlier version of this table read 24.7× at $\mathrm{Re} = 3{\cdot}10^6$ rising to 179× at $\mathrm{Re} = 10^3$, a far more dramatic trend, and it was an artifact of that defect. The direction survived the correction; the magnitude did not, and the corrected magnitude is what is reported.

6.7 What the criterion is for, and what it is not

The sections above establish what a representation does to the near-wall state, and they establish it firmly: the damage is computable, bounded above, ordered correctly across a fifty-fold range of first stations and three and a half decades of Reynolds number, and controlled by station placement rather than by value budget. **It does not follow that this damage is what makes a seed slow, and §5.2.1 shows directly that it is not.**

Three independent attempts to make the near-wall state the mediator all fail:

1. **By placement** (§5.2.1). Moving the first station inside the mesh's first cell takes the wall-gradient error from 1218.8% to 1.8% and the roughness to the converged field's own. Convergence gets *worse*: −266.8% against −181.6%, winning 0 of 5 cases against 1 of 5.
2. **By repair** (§5.5). Inverting a wall function restores a projected seed's gradient from 877.7% to 69.4%. Convergence moves 0.1 points.
3. **By smoothing that repair** (§5.5). Bringing the reconstruction's roughness to 5.7× converged, against the working seed's 4.2×, moves convergence a further 0.6 points.

A seed can reproduce the converged boundary layer to 1.8% in gradient and exactly in roughness and still be among the worst initial conditions measured here. **The near-wall velocity field is not the mediator.**

Where the damage is instead (§5.2.2). Every projection preserves *viscous* drag — the quantity the wall gradient integrates, and 60–84% of the drag — at +69% to +86% against the mesh-native oracle's +92.5%, all 5 of 5 cases. What it destroys is total drag, and therefore pressure drag, at −183.9%. That is consistent with §5.4, where an inconsistently seeded channel destroys `C_d,p` while leaving the shear-driven quantities alone, and with the one arm in this study that is positive on pressure drag being the one whose reconstructed wall shear was smoothed.

So the criterion should be used for what it measures. It says, before any solve, how badly a given output format will misreport the near-wall state, it is conservative, and it orders formats correctly. That is a genuine and cheap diagnostic of a representation. It is **not** a predictor of convergence, and §6.4's pre-flight tool must be read as reporting fidelity rather than forecasting a speedup. The property that does predict convergence here — whether the representation preserves a consistent pressure field — we can locate but not yet compute in advance, and §10 says so.

7. What does not work, and why that matters

Three predictions a reader would reasonably make are false here, and each was tested rather than argued away. They are in the paper because a recipe that only ever confirms itself is not evidence, and because each failure is explained by the same closed form as the successes.

7.1 Refining the raster does not help, and the amount by which it does not is predicted

The obvious response to §5.2 is to spend more values. We measured it — `scripts/resolution_ladder.py`, uniform Cartesian rasters from 128^2 to 421^2 of the **exact converged field**, so surrogate accuracy is not in the comparison — and the saving is flat and negative throughout.

§6 says why, quantitatively. The predicted first-cell gradient overestimate across that ladder is

raster	values	first station	`y+`	predicted `G`
128^2	16,384	$1.17{\cdot}10^{-2}$	1677	36.6×
181^2	32,761	$8.29{\cdot}10^{-3}$	1186	36.6×
256^2	65,536	$5.86{\cdot}10^{-3}$	839	36.6×
362^2	131,044	$4.14{\cdot}10^{-3}$	593	35.9×
421^2	177,241	$3.56{\cdot}10^{-3}$	510	35.3×

A 10.8-fold increase in stored values moves the predicted damage from 36.6× to 35.3×. Two things make it flat. `u+` grows logarithmically, so quadrupling the resolution buys `ln(4)/kappa ~ 3.4` on a value in the twenties; and above the boundary-layer edge the velocity has saturated at freestream, so it buys nothing at all. The measured ladder is flat for the same reason, and the closed form turns "we tried and it did not help" into "here is the factor by which it cannot".

The scale of the gap is worth stating plainly. Resolving one cell across the inner layer on a uniform grid would need N ≈ 11,800 — about 28× beyond what the standard datasets hold and roughly 10^8 stored values. **The uniform-grid route is not expensive; it is closed.**

7.2 Seeding what the cold solver is slowest at makes it slower

Decomposing the cold solve by quantity suggests an obvious strategy. Iterations to settle within 1% of converged, cold against a seed of the exact field:

quantity	cold	oracle seed	share of `C_d`
viscous drag `C_d,v`	~700	~53	60–84%
lift `C_l`	~950	1	—
pressure drag `C_d,p`	~1850	1–2	16–40%

A cold solver is slow at pressure and fast at the near-wall velocity gradient; a surrogate is the reverse. The inference — hand over the pressure, keep the near-wall velocity — is what we pursued, and it is **false**:

- `fitted_p`, pressure only, is **inert**: +0.2% on drag, +0.1% elsewhere, at every depth.
- `composite`, potential-flow pressure plus a surrogate boundary layer, is **−305.4%**.
- `potentialFoam` alone — the free classical alternative — is inert on drag (+0.6% on `C_d`@1%) and mildly positive on lift (+3.3%).
- The arm that wins hands over velocity and eddy viscosity inside the boundary layer and **no pressure at all**.

The reason is SIMPLE's structure [25]. Pressure is recomputed from continuity given the velocity field, so a pressure seed inconsistent with `U` is overwritten within a few iterations; only fields entering the momentum and turbulence transport carry information forward. We keep this section because a falsified prediction from a measured decomposition is stronger evidence than an unfalsified one, and because it is the reason the recipe is not obvious.

7.3 The wake is worth half a per cent here

The largest acceleration reported in this literature — 26.3× iterations, 16.4× wall-clock [8] — comes from initialising the far wake, and every seed in this paper deliberately does the opposite, cutting off at 3.5 chords and handing the wake back to the solver. That looks like a limitation, so we bounded it rather than defending it.

`scripts/wake_probe.py` seeds the **exact converged field** across the whole downstream region — 37.5% of the cells, 21.6% of them fully — which bounds what *any* wake model could buy on these cases. Five cases, Re = $3{\cdot}10^6$:

metric	oracle wake seed	95% CI	per case
`C_d,v`@1%	**+0.5%**	[+0.4, +0.7]	+0, +0, +1, +1, +1
`C_d,v`@0.5%	+1.3%	[+1.0, +1.5]	+1, +1, +1, +1, +2
`C_d`@1%	−242.1%	[−676, −0.3]	−1098, −103, −19, −4, +13
`C_l`@1%	−22.0%	[−68, +1.6]	−92, +0, +2, +2

A perfect wake seed is worth half a per cent. On a 2-D attached-flow airfoil at Re = $3{\cdot}10^6$ on a 20-chord C-grid, the solver is not spending its time developing the wake; it is spending it on the near-wall state. So the two results are about different regimes, and §2.1 notes that the 26.3× is itself reported as conditional on an accurate near-body field being supplied separately. Our restriction to the boundary layer is a **finding**, not a compromise, and their result and ours compose rather than compete.

It also repeats §5.4's lesson at a different scale: the wake seed is *harmful* on total drag, because handing over a downstream field while leaving the boundary layer cold is another inconsistent pair. Consistency is not a detail of the recipe — it is most of it.

7.4 What the criterion does not do

It is necessary, not sufficient, and the study contains its own counterexample. `nf_mesh` hands over the network's whole-field prediction at the solver's cell centres: it satisfies the placement criterion perfectly, retains the wall gradient, and is the **worst arm in the study** at below −568% on total drag, because the model's training `sdf` distribution is centred on 0.23 chords while the C-grid reaches 20, so the outer field is extrapolation. A representation that fails the criterion can be ruled out for free; one that passes it still has to satisfy conditions 2 and 3.

8. An acceptance test that bounds the worst case

Warm starting is only adoptable if a bad seed cannot cost more than not seeding. Ours can: ungated across 5 cases x 15 strategies (70 seeds, every arm in the tree, not a favourable subset), the mean is -163.6% on Cd@1% and the worst single seed is -1169.6%. Only 24 of the 70 seeds help at all.

The rule. Run K probe iterations from the seed. Read two scalars from the residual history — the level log10 r_K and the drop log10 r_K - log10 r_0. Either continue the solve, or discard the seed and start cold. Accepting costs nothing extra, because the probe iterations *are* the first K iterations of the warm solve; rejecting costs K + cold. The worst case is therefore (1 + K/N_cold) x cold **by construction**, whatever the seed does.

The rule never observes a cold run. In production there isn't one — that is the entire point of warm starting. The threshold is calibrated leave-one-case-out and applied to the held-out case.

K = 25 (~3% of a cold solve), threshold on the residual level, scored over every arm in the `repr3` tree:

metric	seeds	ungated mean	ungated worst	**gated mean**	**gated worst**	harmful admitted
Cd@1%	70	-163.6%	-1169.6%	**+1.5%**	**-5.8%**	**0 / 46**
residual 5e-6	70	-161.9%	-1449.3%	-0.5%	**-7.6%**	**0 / 53**
Cd_v@1%	70	+20.1%	-8.6%	+20.1%	-8.6%	14 / 15
Cl@1%	56	+1.1%	-672.6%	**-8.2%**	**-672.6%**	5 / 18

Read the two columns that matter as a pair. The gated mean is small — the gate is insurance, not a profit centre, and selling it as a mean saving would be selling the wrong product. What it does is convert a −1169.6% tail into a −5.8% one on drag, and a −1449.3% tail into −7.6% on the residual, while admitting **none** of the harmful seeds in either case. On viscous drag, where 55 of 70 seeds already help, it is a near no-op at 96% capture.

Where the gate fails, stated rather than buried. On lift it admits 5 of 18 harmful seeds, and its gated worst case is −672.6% — the *same* as the ungated worst, meaning it admits the single worst lift seed in the study. Lift converges by a different route — it is pressure-dominated, and §5.2 showed that resampling *helps* lift while destroying drag — so a probe reading the momentum residual is weakly informative about it. The gate should be applied per quantity, and on the quantity a user cares about; we do not claim it is a universal filter. Its worst-case bound of (1 + K/N) x cold survives regardless, because that bound is arithmetic, not statistical: it holds for any seed, any metric, and any threshold, including a threshold that admits every seed.

The gate is not what makes warm starting fast; it is what makes it deployable. It is conservative by construction, capturing only 12% of what a gatekeeper with foreknowledge would achieve on total

drag — the metric where two thirds of the seeds are harmful. Longer probes are monotonically worse, and not marginally: at K = 400 on drag the rule admits 13 harmful seeds it rejected at K = 25 and returns −43.9%, because the probe cost alone (bound −49.8%) exceeds anything the decision can recover. **A short probe is not a compromise forced by cost; it is the better rule.**

9. Wall-clock

Iterations are the honest unit for a mechanism, but the claim an engineer cares about is seconds — and a mesh-native seed costs seconds a projected one does not: wall-distance computation, surrogate inference, masking, and writing the case.

`scripts/wallclock_control.py` charges each preparation stage to the arms that need it and reports **end-to-end** seconds. It runs serially and refuses to start while any other solver is up; that refusal is part of the measurement. Five cases, all arms, `exclusive: true`.

arm	iterations	**end-to-end seconds**
`oracle_mesh` (control)	+92.0%	**+93.1%**
`nf_bl`	**+34.0%**	**+28.8%**
`fitted_bl`	−190.7%	−138.0%
`cartesian_128`	−513.5%	−308.4%

The iteration saving survives translation into seconds, and the cost of the translation is about five percentage points. That is the transferable finding here, and it holds case by case rather than only on the mean:

case	iterations	seconds	gap
`naca0012@4`	−3.3%	−3.4%	0.1
`naca2412@2`	+40.9%	+34.7%	6.2
`naca0015@6`	+27.6%	+21.5%	6.1
`naca0012@0`	+40.1%	+34.5%	5.6
`naca2415@5`	+64.9%	+56.9%	8.0

Seed construction is charged in full and is small against the solve: backbone inference at 31,700 points **10.4–10.6 s**, `wall_distance` 0.4 s, masking 0.4 s — **~11 s** against cold solves of 88–239 s. The rest of the five-point gap is the per-iteration penalty of a seeded solve, which is modest.

Two details worth stating because they cut *for* us and could look like errors: the **oracle does better in seconds than in iterations** (+93.1% vs +92.0%), and the Cartesian arm is **less bad** in seconds than in iterations (−308% vs −514%). Both have the same cause — a solve started near the answer has cheaper inner linear solves — and both mean iteration counts are the conservative unit.

Readability, stated rather than assumed. This tree carries five arms, not the fifteen of the mechanism study, so its reference is a median over fewer settled arms and its spreads are wider: two of the five cases (`naca2412@2` at 0.751%, `naca0012@0` at 0.745%) exceed the 0.5% limit for a 1% band. On the three readable cases the result is **+29.7% iterations → +25.0% seconds**, a 4.8-point translation cost — the same conclusion at a smaller n. The per-case gap column above is what we actually lean on, and it is independent of the readability verdict entirely.

The saving therefore survives the accounting an engineer would actually do: seconds, on one machine, with the seed's own construction charged to it.

10. Limitations

On the criterion and the closed form

- **The closed form assumes an equilibrium wall profile.** It uses the law of the wall with standard smooth-wall constants ($\kappa = 0.41$, $B = 5.0$), unmodified. Under strong adverse pressure gradients, separation, roughness, compressibility or heat transfer the profile departs from it, and the predicted factor should be read as indicative rather than as the 13%-accurate number it is on the attached cases measured here.
- **It is quantitative only while the representation's first station lies inside the boundary layer.** Above that the velocity has saturated at freestream and the expression becomes an upper bound; we report the regime alongside every number rather than leaving a reader to infer it.
- **It measures representations; it does not forecast solves** (§6.7). This is the paper's largest limitation and it is a negative result rather than an unexplored gap. Three independent routes to a correct near-wall state — by grading (§5.2.1), by wall-function repair and by smoothing that repair (§5.5) — all leave convergence unchanged or worse. So the pre-flight check of §6.4 reports how badly a format will misreport the near-wall state, and **nothing follows from it about the speedup**. Use it to rule a format out, never to predict a gain.
- **We locate the damage but cannot yet compute it in advance.** §5.2.2 shows a projection preserves viscous drag and destroys the pressure field, which is consistent across every projected arm and with §5.4's channel result. We have no closed form for *that*, and no pre-flight test for it. Finding one is the obvious continuation of this work and we do not claim to have done it.
- **Pressure drag is a secondary quantity here and is treated as one.** It converges three times slower than total drag from cold, its 1% row is unreadable on these trees, and the smoothed-repair observation on it (§5.5) was found after the fact. It is reported as an observation, not a result.
- **It is necessary, not sufficient** (§7.4). `nf_mesh` retains the gradient perfectly and is the worst arm in the study, so even as a veto the check has to be read alongside the region and channel conditions.
- **It is a bound, not an estimate, and it over-predicts by 1.3–2.6×** (§6.2). We do not absorb that into a fitted coefficient: fitting it would turn a prediction into a description of these five cases. An earlier version of this work reported 13% agreement; that figure was measured against a projection with a wall-distance bug and is withdrawn.

On the repair, which did not work

- **We do not know why it failed**, and §5.5 says so. The repair restores the first-cell gradient to 69.4% error, close to the mesh-native seed's 53.7%, and the convergence saving does not follow. The one measurable difference we can point at is that the repaired seed is 11.1× rougher along the wall than the converged field against 4.2× for mesh-native, but **that is a candidate, not a finding**, and a tangentially-smoothed repair is the experiment that would settle it.
- **The repair assumes an equilibrium profile**, as the closed form does, and assumes the representation's first station carries a usable velocity. Where the layer separates the inverted `u_tau` is meaningless. Every case here is attached to incipiently separated, so the negative result is established only in the regime where the repair should have been at its best — which makes it a

stronger negative, not a weaker one.
- **It rebuilds `nut` from a damped mixing length**, which is what Spalart–Allmaras relaxes to in the log layer but not what SA solves, and §5.4 shows `nut` is the least forgiving channel. We cannot exclude that this, rather than the gradient reconstruction, is what the solver objected to.

On the study

- **2-D, incompressible, steady, one turbulence model.** Spalart–Allmaras only. Whether the three conditions survive a two-equation model is untested, and k-ω SST is the obvious next experiment.
- **One solver and one mesh family.** OpenFOAM SIMPLEC on a C-grid we generate. Nothing here has been tried on an unstructured or commercial solver.
- **One Reynolds number for the *solver* results** ($\mathrm{Re} = 3{\cdot}10^6$). The convergence savings, the conditions and the repair are all measured there and nowhere else. The *mechanism* is checked across $\mathrm{Re} = 10^3$ to $3{\cdot}10^6$ in §6.6, but that check is on the seed's wall gradient, not on convergence: it does not show that a warm start helps at another Reynolds number, only that the damage a representation does behaves as the closed form says it will.
- **The closed form is accurate only where the boundary layer is turbulent and attached.** §6.6 measures agreement of 0.83–0.96 at $\mathrm{Re} \geq 10^6$ and 0.32–0.35 at $\mathrm{Re} \leq 10^5$, where 42–62% of the surface carries near-zero wall shear. There it is a lower bound rather than an estimate — conservative, but not quantitative.
- **Bands below 1% need a longer budget.** At 0.5% and 0.2% the total-drag rows are unreadable at 6000 iterations on the corpus.
- **`nuTilda` is floored at freestream on write**, a common-mode limitation quantified in §5.8 — it removes 2.1–2.3% of the eddy-viscosity field's energy in the boundary layer and applies identically to every arm including the oracle.
- **Three cases have no unique steady drag at this budget**, and they share a shape: `naca4412@3deg`, `naca4415@2deg`, `naca4415@4deg` — thick cambered sections at low incidence, carrying the corpus's worst residual floors. We do not know whether this is genuine non-uniqueness or a budget we did not pay, and we do not claim to.
- **The residual metric is negative for the recommended arm** (§5.6). We report it every time we report the force metric.
- **Wall-clock is n = 5 and was measured exclusively; the placement, repair and sequencing trees were not.** Those three were run concurrently, so iterations from them are sound — iteration counts are contention-proof — and **no wall-clock number is quoted from them.**
- **The acceptance test fails on lift** (§8): its gated worst case equals its ungated worst case, so it admits the single worst lift seed in the study. The `(1 + K/N)` bound is arithmetic and survives that, and §7.2 explains why lift behaves differently from drag.

11. Conclusion

Whether a neural surrogate can accelerate a production RANS solve is decided by its **output representation**, not by its accuracy. The cleanest statement of that needs no network at all: take the exact converged flow field and hand it to the solver three ways. Read at the solver's own cell centres it saves **92.0%** of a cold solve. Stored first on an equal-budget wall-fitted grid it costs **181.6%** (95% CI −318 to −45, winning 1 case of 5). Stored on a uniform 128^2 raster, **548%**. One field, three representations, a 274-point swing.

What a representation does to the near-wall state we can state in closed form. A resampled field hands every cell nearer the wall than its first station that station's value, so the first-cell wall gradient is overestimated by `u+(y1+)/u+(y_c+)` — no fitted parameter, an upper bound on the measurement across a fifty-fold range of stations and from $\mathrm{Re} = 10^3$ to $3{\cdot}10^6$. It indicts placement rather than budget: moving the first station inside the mesh's first cell takes the gradient error from 1218.8% to **1.8%**, and does it at half the stored values as readily as at the full budget.

And that is not what costs the solve. The seed reproducing the converged boundary layer to 1.8% in gradient, and exactly in roughness, converges at −266.8% and wins none of five cases — worse than the seed carrying 1218.8% error. Repairing the gradient by wall function moves convergence by a tenth of a point; smoothing that repair to the working seed's own roughness moves it by half a point more. Three independent routes to a correct near-wall state, and none of them recovers the solve.

Where the damage is instead, the decomposition says plainly: every projection preserves **viscous** drag — 60–84% of the drag, and the quantity the wall gradient integrates — at +69% to +86% against the mesh-native oracle's +92.5%, five cases of five. What it destroys is the **pressure** field, at −183.9% on `C_d,p`. The near-wall velocity survives a round trip well enough for wall shear; the pressure field does not survive it at all.

Read as advice rather than as a result, the paper is short:

Query your surrogate where the solver lives. Not because a grid cannot hold the boundary layer — with the right grading it can, to 1.8% — but because storing the field on a grid at all corrupts something the boundary layer does not contain. Give the solver only the region your surrogate is trusted on, hand over whole physics rather than single channels, and spend 3% of a solve checking before you commit.

The size of the effect is modest and we say so. The recommended seed accelerates viscous-drag convergence by 18.4% across thirteen cases, winning every one of them ($p = 0.0002$), with a converged-field control at 93.6% and a null negative control. That is far short of the 26.3× reported elsewhere for a different regime. What is durable here is not the number.

What is durable is that the representation result needs no network to state, and that the paper's own explanation of it was tested to destruction rather than asserted. We predicted the near-wall gradient was the mediator, registered the prediction before running it, built three separate ways to restore that gradient, and reported that all three failed. The criterion that survives is a diagnostic of what a representation keeps — cheap, conservative, and correct in its ordering — and not a forecast of a speedup. Naming the difference between those two is, we think, the more useful contribution.

CRediT authorship contribution statement

Ali Jabbary: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing, Supervision. **Kasra Ghanavati:** Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All code, meshes, solver configurations and result files are openly available at `https://github.com/ali-kin4/neuroforge-cfd`. Every table and figure in this paper is regenerated by a single command from checkpointed solver output; Appendix A gives the mapping from each table to the script and result file that produces it. The flow solver is OpenFOAM v2606 (ESI), used unmodified. The surrogate is trained on the public AirfRANS dataset.

Appendix A – reproduction

Each result maps to one script and one committed result file.

result	script	result file
Wall-gradient diagnostic (§6)	`seed_gradient_diagnostic.py`	`seed_gradient.json`
Three conditions (§5.2–§5.4)	`mesh_native_probe.py`	`depth_repr3_nowake.json`
Thirteen-case corpus (§5.1)	`corpus_probe.py`	`depth_corpus.json`
Acceptance certificate (§8)	`certificate.py`	`cert_all13_*.json`
Wall-clock (§9)	`wallclock_control.py`	`wallclock_control.json`
Wake bound (§7.3)	`wake_probe.py`	`wake_probe_*.json`
Figures 1–2	`plot_mechanism.py`, `plot_bands.py`	`mechanism.png`, `bands.png`

Re-scoring any finished tree at every convergence depth and force band is a single command, `reanalyse_depth.py`, which also declares the arm set it scored over and prints the readability verdict for every row.

Appendix B – the scoring rules as code

The eight rules of §4 are implemented in `solver/scoring.py` and `scripts/reanalyse_depth.py` as `has_settled`, `settled_reference`, `bounded_saving`, `shared_reference`, `reference_spread`, `readable_depth`, `scoreable_budgets`, `bootstrap_ci` and `sign_test`, with `MIN_DEPTH_OVER_FLOOR = 5.0` and `MAX_SPREAD_FRACTION = 0.5`. Each carries a unit test named for the mistake it prevents.

References

[1] F. Bonnet, A. J. Mazari, P. Cinnella, P. Gallinari. AirfRANS: high-fidelity computational fluid dynamics dataset for approximating Reynolds-averaged Navier–Stokes solutions. Advances in Neural Information Processing Systems 35 (2022). arXiv:2212.07564.

[2] H. Wu, H. Luo, H. Wang, J. Wang, M. Long. Transolver: a fast transformer solver for PDEs on general geometries. International Conference on Machine Learning (2024). arXiv:2402.02366.

[3] C. Zeng, Y. Zhang, J. Zhou, et al. Point cloud neural operator for parametric PDEs on complex and variable geometries (2025). arXiv:2501.14475.

[4] M. S. Eshaghi, C. Anitescu, N. Valizadeh, Y. Wang, X. Zhuang, T. Rabczuk. NOWS: neural operator warm starts for accelerating iterative solvers. Computer Methods in Applied Mechanics and Engineering 458 (2026) 118989. arXiv:2511.02481.

[5] J. Oh, Y. Lee, J. Darbon, et al. Spectrally safe neural operator warm-starts for large-scale Newton solvers (2026). arXiv:2606.21828.

[6] M. Khodak, M. K. Jung, B. Wynne, et al. One-shot acceleration of transient PDE solvers via online-learned preconditioners (2025). arXiv:2509.08765.

[7] P. Sharpe, R. Ranade, K. Tangsali, et al. Accelerating transient CFD through machine learning-based flow initialization (2025). arXiv:2503.15766.

[8] K. W. Fuchi, E. M. Wolf, C. R. Schrock, P. S. Beran. Acceleration of RANS solver convergence via initialization with wake extension models (2025). arXiv:2501.14699.

[9] I. Yavlovich, J. Agbaria, M. Mhamed, et al. Learning-augmented scalable linear assignment problem optimization via neural dual warm-starts (2026). arXiv:2605.09382.

[10] P. R. Spalart, S. R. Allmaras. A one-equation turbulence model for aerodynamic flows. La Recherche Aerospatiale 1 (1994) 5–21.

[11] H. G. Weller, G. Tabor, H. Jasak, C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. Computers in Physics 12 (6) (1998) 620–631.

[12] X.-H. Zhou, J. Han, M. I. Zafar, E. M. Wolf, C. R. Schrock, C. J. Roy, H. Xiao. Neural operator-based super-fidelity: a warm-start approach for accelerating steady-state simulations. Journal of Computational Physics (2025). arXiv:2312.11842.

[13] T. Hu, C. Wu, J. Ding, X. Wang, Y. Yang, J. Wang. Data-driven flow initialization framework for CFD acceleration of underwater vehicle in vertical-plane oblique motion (2026). arXiv:2601.02693.

[14] T. A. Mehta, P. S. Bhati, H. D. Akolekar. DD-RNO: a domain-decomposed routed neural operator for airfoil flow prediction (2026). arXiv:2608.13490.

[15] X. Zhang, Y. Huang, S. Jiang, et al. Geometry-aware anisotropic boundary correction for aerodynamic simulation (2026). arXiv:2606.09963.

[16] Z. Yang, H. Xin, T. Du, et al. Simple yet effective: low-rank spatial attention for neural operators (2026). arXiv:2604.03582.

[17] Z. Zhang, X. Yang, Y. Miao, et al. PGOT: a physics-geometry operator transformer for complex PDEs (2025). arXiv:2512.23192.

[18] E. J. Schmidtobreick, D. Arnstroem, P. Haeusner, et al. Warm-starting active-set solvers using graph neural networks (2025). arXiv:2511.13174.

[19] P. Araujo da Cunha Sousa, A. M. Afonso, C. Veiga Rodrigues. Surrogate-based pressure–velocity coupling: accelerating incompressible CFD flow solvers with machine learning. Computers & Fluids (2026), available online July 2026.

[20] J. Scherz, D. Hines, P. Bekemeyer. Evaluation of state-of-the-art deep learning architectures for aerodynamical predictions (2026). arXiv:2607.13866.

[21] M. Yagoubi, D. Danan, M. Leyli-Abadi, et al. NeurIPS 2024 ML4CFD competition: results and retrospective analysis (2025). arXiv:2506.08516.

[22] K. W. Fuchi, E. M. Wolf, D. S. Makhija, et al. Multi-fidelity machine learning applied to steady fluid flows (2025). arXiv:2501.14870.

[23] Y. Liu, H. Wang, Y. Qi, et al. Full-field prediction for engineering-scale three-dimensional aircraft with multigrid-hierarchical learning (2026). arXiv:2605.30375.

[24] Y. Yang, B. Gholami, C. Gurbuz, et al. Towards a foundation-model paradigm for aerodynamic prediction in three-dimensional design (2026). arXiv:2604.18062.

[25] S. V. Patankar, D. B. Spalding. A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. International Journal of Heat and Mass Transfer 15 (10) (1972) 1787–1806.

[26] Y. Liu, W. Zhang, J. Kou. Mode multigrid – a novel convergence acceleration method (2018). arXiv:1802.08962.